



Proyecto con uso de Framework

React: HormigaLibre

Nombres: Ignacio Macaya - Jared Manzo

Asignatura: Programacion FrontEnd

Docente: Sebastián Alejandro Pizarro Álvarez

Sección: D-FB50-N3-P13-C2

Nombre del proyecto: HormigaLibre

Descripción de la Problemática: Con el escenario económico actual, la gestión de las finanzas personales representa un gran desafío, principalmente para jóvenes y estudiantes de educación superior. Uno de los factores mas importantes al momento de desestabilizar el presupuesto mensual limitado que pueden tener los jóvenes son los denominados “Gastos Hormiga”, estos son gastos que al tener bajo valor unitario no se toman muy en cuenta pero que al paso de los días van creando un efecto de bola de nieve y se transforma en quizá decenas de pequeños gastos que juntos constituyen una gran cantidad del capital disponible disminuyendo enormemente la capacidad de ahorro para estas personas.

ODS relacionados al proyecto:

ODS 8: Trabajo Decente y Crecimiento Económico

- Justificación: Una de las metas fundamentales de este objetivo es promover la resiliencia económica y el crecimiento financiero sostenible en las comunidades y las personas. Los gastos hormiga representan un obstáculo directo para la estabilidad financiera personal y familiar, ya que erosionan la capacidad de ahorro y propician situaciones de sobreendeudamiento temprano.

ODS 12: Producción y Consumo Responsables

- Justificación: Este objetivo busca garantizar modalidades de consumo sostenibles mediante la toma de decisiones informadas y la reducción del desperdicio de recursos. Los gastos hormiga están íntimamente ligados a patrones de consumo impulsivo, innecesario y poco planificado (tales como la compra desmedida de snacks procesados, productos con plásticos de un solo uso o la mantención de servicios digitales en desuso).

Público Objetivo: El sistema está diseñado metodológicamente para abordar las necesidades de dos perfiles de usuarios principales dentro del ecosistema urbano:

- **Estudiantes de Educación Superior:** Jóvenes universitarios o de institutos profesionales que cuentan con presupuestos mensuales limitados, dependientes de asignaciones familiares, becas o empleos de tiempo parcial. Este segmento es altamente vulnerable a los gastos hormiga debido a las dinámicas de consumo asociadas a la vida estudiantil (compras rápidas entre clases, colaciones no planificadas y snacks).
- **Jóvenes Profesionales y Trabajadores Iniciales:** Personas que se encuentran insertándose en el mercado laboral y experimentando sus primeros ingresos independientes. Al no poseer un hábito consolidado de contabilidad personal, suelen subestimar los microgastos de transporte, alimentación exógena o suscripciones automáticas, viendo mermada su capacidad real de ahorro mensual.

Funcionalidades:

Gestión Completa del Ciclo de Datos (Operaciones CRUD)

- **Creación de Registros (Create):** Permite al usuario ingresar un nuevo gasto hormiga capturando de forma estructurada el concepto o detalle, el monto en pesos chilenos, la categoría del consumo y la fecha en que se efectuó. Al procesarse, el sistema genera de manera interna un identificador único (ID seguro) para el elemento.
- **Visualización Dinámica (Read):** Muestra el historial completo de los gastos registrados de forma cronológica y ordenada en tarjetas independientes, extrayendo la información directamente del estado de la aplicación.
- **Actualización de Datos (Update):** Permite al usuario corregir o modificar cualquier registro existente. Al presionar el comando de edición, el formulario absorbe de manera reactiva los datos del gasto seleccionado, permitiendo guardar los cambios e impactar la persistencia sin alterar el identificador original del objeto.
- **Eliminación Controlada (Delete):** Proporciona un mecanismo para remover registros del historial de forma selectiva mediante funciones de filtrado inmutable del estado, incluyendo una ventana de confirmación nativa para evitar pérdidas accidentales de datos.

Diseño inicial de componentes:

- **Componente App (Gestor Raíz y Datos):** Encargado de centralizar el estado global de la aplicación (gastos y filtroCategoría). Administra el ciclo de vida de los datos mediante `useEffect` para garantizar la persistencia automática en Local Storage y distribuye las funciones controladoras (CRUD) hacia los componentes hijos.
- **Componente Navbar (Control de Búsqueda y Métricas):** Actúa como la barra superior de la interfaz. Integra el selector global de categorías, permitiendo al usuario filtrar todo el historial visualizado, y realiza en tiempo real el cálculo acumulado del gasto total en pesos chilenos.
- **Componente GastoForm (Formulario Reactivo de Entrada):** Interfaz especializada en la captura de datos para los procesos de Creación y Actualización. Controla los estados locales de los inputs (Detalle, Monto, Categoría, Fecha) e implementa las capas de validación y desarrollo seguro antes de interactuar con el estado global.
- **Componente GastoList (Contenedor del Historial):** Componente estructural encargado de recibir el arreglo de datos filtrado. Ejecuta de forma limpia el método `declarativo.map()` para iterar la información y estructurar la visualización general de la interfaz.
- **Componente GastoItem (Visualizador Reutilizable de Registros):** Componente atómico de presentación que recibe las propiedades de un único gasto. Renderiza por categoría y expone los controladores interactivos para activar las operaciones de Edición y Eliminación del CRUD.

Evidencias de uso de IA.

Caso de Uso 1: Definición de ODS y Arquitectura Modular

- Herramienta utilizada: Gemini (Google AI).
- Propósito de uso: Identificar el impacto sostenible del proyecto mediante los Objetivos de Desarrollo Sostenible de la ONU y estructurar la jerarquía inicial de componentes en React.
- Pregunta realizada:

"Mi proyecto de React sería una app web para el cálculo de gastos hormiga, quiero saber en qué ODS se puede basar este proyecto, necesito que sean al menos 2 y quiero el diseño de la página web para mi proyecto de React."

- Resultado obtenido: La IA identificó con precisión el ODS 8 (Crecimiento Económico) y el ODS 12 (Consumo Responsable), entregando sus respectivas justificaciones teóricas. Adicionalmente, propuso una estructura de componentes basada en un flujo unidireccional compuesto por App, Navbar, GastoForm, GastoList y GastoItem.
 - Respuesta: "¡Excelente idea! Tu proyecto se alinea perfectamente con el ODS 8 (Trabajo decente y crecimiento económico) al promover la salud financiera y la capacidad de ahorro, y con el ODS 12 (Producción y consumo responsables) al fomentar hábitos de consumo conscientes y reducir compras impulsivas innecesarias. Para la estructura en React, te sugiero un diseño modular compuesto por: App (estado global), Navbar (resumen financiero), GastoForm (formulario de registro) y GastoList (historial de consumos)."
- Decisión tomada por el equipo: El equipo evaluó las sugerencias conceptuales y técnicas, decidiendo adoptar formalmente ambos ODS para el marco conceptual del informe e implementar la arquitectura modular propuesta para la separación de archivos en la carpeta src/

Caso de Uso 2: Reestructuración de Filtros Globales en la Barra de Navegación

- Herramienta utilizada: Gemini (Google AI).
- Propósito de uso: Resolver la comunicación entre componentes distantes para permitir que un control de interfaz superior (Navbar) filtre colecciones de datos dispuestas en un listado inferior.
- Pregunta o instrucción realizada (Prompt):

"Quiero que mi navbar sirva para buscar gastos por categoria"

- Resultado obtenido: La IA guió al equipo en la implementación del patrón arquitectónico en React. Explicó cómo mover el estado del filtro desde GastoList hacia App.jsx, permitiendo que el Navbar despache el nuevo criterio de búsqueda mediante funciones callback y que la lista reaccione de manera puramente declarativa mediante métodos síncronos de filtrado.
 - Respuesta: ""Para lograrlo, debes aplicar la técnica de Lifting State Up (Elevación del Estado). Mueve el estado filtroCategoria de la lista al componente principal App.jsx. Luego, pásale ese estado y la función para cambiarlo al Navbar mediante props. Finalmente, pasa el estado filtrado a GastoList para que aplique un .filter() de forma síncrona antes de renderizar las tarjetas."
- Decisión tomada por el equipo: Tras comprender la ventaja de centralizar los estados para la comunicación limpia entre componentes hermanos, el equipo eliminó los controles redundantes del listado y unificó la experiencia de búsqueda directamente en el encabezado de la app.

Caso de Uso 3: Persistencia de Datos con Local Storage e Inmutabilidad

- Herramienta utilizada: Gemini (Google AI).
- Propósito de uso: Implementar un mecanismo para que los gastos registrados no se borren al refrescar el navegador, cumpliendo las reglas de integridad de datos.
- Pregunta o instrucción realizada (Prompt):

"¿Cómo hago para que los gastos no se borren cuando el usuario recargue la página en React?"

- Respuesta de la IA (Acotada):

"Debes usar la API de Local Storage combinada con useEffect. Inicializa tu estado gastos con una función que intente leer e interpretar la clave mediante `JSON.parse(localStorage.getItem('gastos'))`. Luego, añade un useEffect que escuche los cambios en el array gastos y guarde automáticamente la lista actualizada usando `localStorage.setItem('gastos', JSON.stringify(gastos))`."

- Decisión tomada por el equipo: El equipo implementó la lógica sugerida dentro de App.jsx. Esto garantizó la persistencia obligatoria exigida en la pauta y permitió verificar que la consistencia de los datos se mantuviera intacta a lo largo de las pruebas de navegación.

Caso de Uso 4: Diseño Seguro y Validaciones del Formulario

- Herramienta utilizada: Gemini (Google AI).
- Propósito de uso: Agregar capas de seguridad al formulario para evitar que se guarden datos nulos, corruptos o inconsistentes.
- Pregunta o instrucción realizada (Prompt):

"¿Qué validaciones de seguridad básicas le puedo poner al formulario de gastos en React antes de enviarlo?"

- Respuesta de la IA (Acotada):

"Antes de ejecutar tus callbacks de guardado, intercepta el evento con `e.preventDefault()`. Aplica `.trim()` al detalle para evitar strings con espacios vacíos y verifica que todos los campos requeridos existan. Crucial para la lógica financiera: valida que el monto sea convertido a número y que sea estrictamente mayor a cero (`Number(monto) > 0`), mostrando un estado de error local si falla."

- Decisión tomada por el equipo: Se añadieron las validaciones propuestas en el componente `GastoForm.jsx`. Esto previno el ingreso involuntario de datos inconsistentes en el sistema de almacenamiento local y aseguró el cumplimiento del criterio de desarrollo seguro del proyecto.